

RelaySMS

SMSWithoutBorders

Technical paper

Afkanerd

January 2026

Version 1.1

Table of Contents

Terminology	2
Introduction	
Background & SMS Ecosystem	3
Background	3
SMS Ecosystem	3
Architecture	5
Infrastructures	5
Clients (Android & iOS app)	5
Vault	6
Gateway client	6
Gateway server	6
Platform publisher	7
Bridge publisher	7
Authentication and key exchange	7
Long Lived Tokens (LLT)	9
Device ID (DID)	11
Encryption and data flow	11
Encryption	11
Data flow	15
Bridges	16
Threat model	21
Blocking Gateway Client numbers	21
Anonymity	21
Harvest Now, Decrypt later attacks	21
Limitations	22
Conclusions	22
References:	23

Terminology

HASH: This almost always implies using SHA-256. It would be stated if another algorithm is used

ENCRYPT: This implies an AES-256-GCM implementation

DR: Double Ratchet implementation as specified by the Signal protocol

AD: Associated data - This is the associated data used during encryption

Introduction

Background & SMS Ecosystem

Background

RelaySMS is a transport layer that bridges SMS with online platforms, allowing users to securely communicate across both. RelaySMS is part of the SMSWithoutBorders project which began in 2021. It works best in the context of a complete internet shutdown; SMS messaging is amongst the only available alternatives to communicating long distance.

RelaySMS relies on the users using their online accounts in publishing messages online. It also provides aliases for users who intend to publish using a phone number without using their online accounts. Being an open source project, the aim is for more security and technically oriented users to host their own versions.¹ We are taking steps to make the clients more suitable for custom hosting and not require developers fully rebuild the entire app to use with customly hosted servers.

This project is funded by the Open Technology Fund.

SMS Ecosystem

(source: wikipedia)

SMS developed in 1993 has come a long way as a messaging service provided by mobile phone operators. The service allows users to send and receive text messages of up to 160 characters. Messages are sent with the MAP MO- and MT-ForwardSM operations, whose payload length is limited by the constraints of the signaling protocol to precisely 140 bytes ($140 \text{ bytes} \times 8 \text{ bits / byte} = 1120 \text{ bits}$)[2]. Larger content (concatenated SMS, multipart or segmented SMS, or "long SMS") can be sent using multiple messages, in which case each message will start with a User Data Header (UDH) containing segmentation information. Since UDH is part of the payload, the number of available characters per segment is lower: 153 for 7-bit encoding, 134 for 8-bit encoding and 67 for 16-bit encoding[3]. The receiving handset is then responsible for reassembling the message and presenting it to the user as one long message.

¹ <https://github.com/smswithoutborders>

Encryption increases the size of the user's message; this includes sending an IV and Digest alongside each payload. Each payload has meta data which requires ~12 bytes (after base64 encoding) and each header for the DR requires ~80 bytes (IV = 16 bytes, serialized ciphertext for header = 32 bytes, HMAC digest appended = 32 bytes). When Device ID is used, another 32 bytes for the key size is appended to the message. This brings the meta information required for sending a message up to 92 (12 + 80) bytes and 124 (12 + 80 + 32) bytes with device ID.

Sending a single message from RelaySMS requires at least 1 extra segment which would be used for transmitting the meta information. The calculations for how much SMS message required becomes:

NumberOfSegments = Number of ASCII characters / 140

Number of segments is rounded up to the nearest integer

Total number of segments = 1 + NumberOfSegments

RelaySMS takes these constraints into design consideration as each SMS message would cost the user money. While the amount of money is less (on average) than what the user would spend on an internet connection², the size and richness of the content being sent is less than what could be sent over an internet connection.

² *In some developing parts of the world*

Architecture

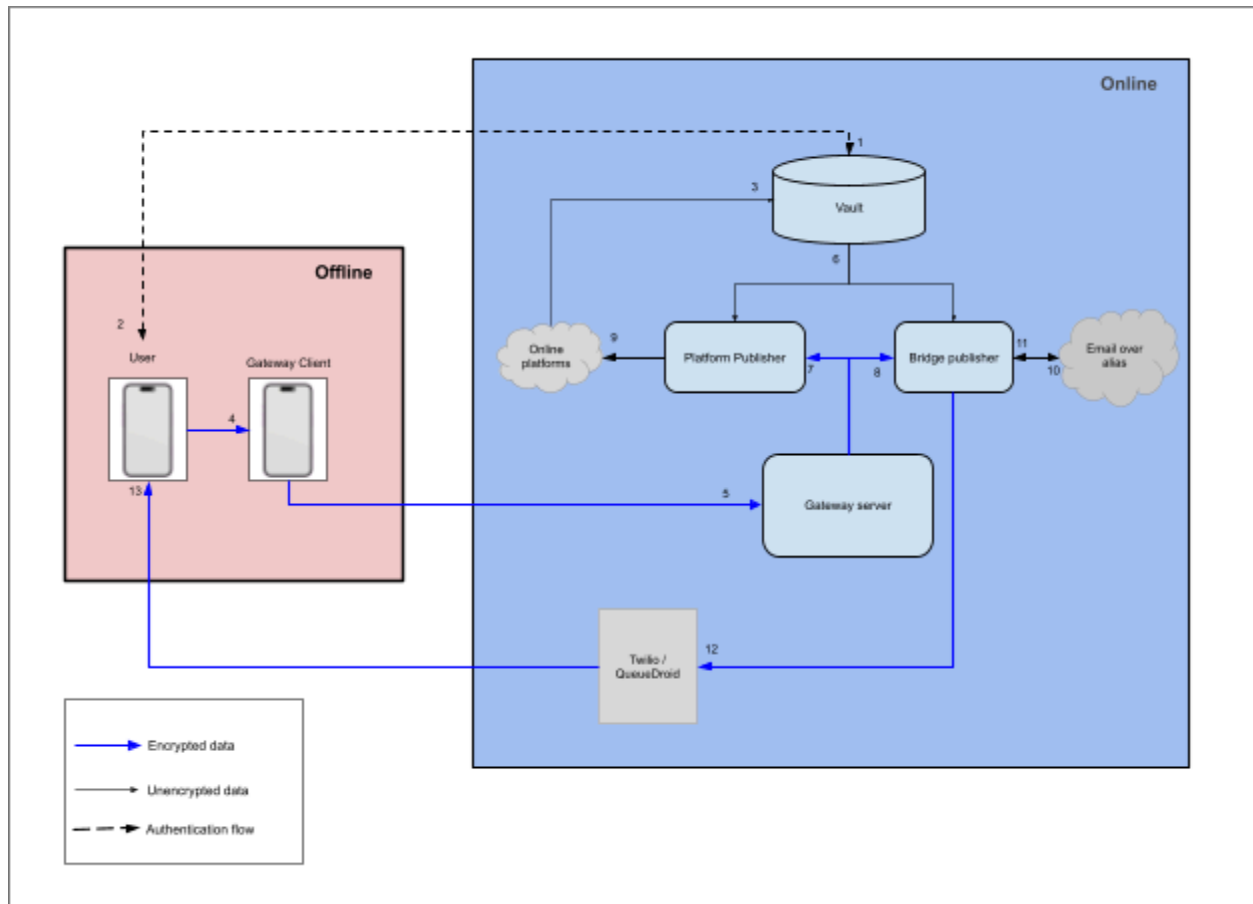


Figure 1.0

Infrastructures

Clients (Android & iOS app)

The clients provide the entire user interface for the service for the user. They provide users with the capability to compose and send out secured SMS messages and manage their accounts. The officially supported clients are the RelaySMS Android and iOS apps.

Role:

The value of the clients is the availability of the Keystore/Keychain for securely storing secrets and private keys which they provide. They also have inbuilt modems given the ability to send out SMS messages from the device. They are also portable and provide the user with the ability to use variant SIM cards.

Vault

The Vault is the central piece of software on the server for storing and accessing user OAuth 2.0 tokens for the user. Communication with the Vault is done through gRPC with both internal (used by locally hosted units like the publishers) and publicly available methods (available to the clients). The client encrypted payloads are terminated here with the Vault being responsible for securely storing the states for each user it communicates with.

Role:

The Vault provides secure storage for the user's OAuth 2.0 tokens, and manages the server side Double Ratchet states used for decrypting and encrypting the incoming and outgoing payloads.

Gateway client

The Gateway clients are Android devices tasked with receiving incoming SMS messages and forwarding them to the cloud. This can be done with either supported protocol such as HTTPS, SMTP and FTP; the reason for the protocols is to be resilient in case of a protocol blockage. The currently officially supported Gateway client app is the open source DekuSMS³. The Gateways clients are built to have queuing functionalities with connectivity listeners; this means the apps queue the incoming messages until the Gateway client is connected to the internet to resume forwarding. .

Role:

This acts as a bridge between the offline users and the online platforms by receiving and forwarding the incoming SMS messages to the server.

Gateway server

The Gateway Server is a piece of software in the cloud responsible for receiving SMS messages forwarded by Gateway clients and routing them to the appropriate processing component (platform or bridge) based on the message type. Communication with the Gateway Server is performed over supported

³ Those hosting their own versions can use whichever SMS forwarding mechanism deemed preferable

protocols such as HTTP, SMTP, and FTP, allowing Gateway clients to forward messages reliably even under network or protocol restrictions.

Role:

The Gateway server is responsible for processing the incoming messages from Gateway clients and deciding which internal component needs to process it. This isolates the publishing units and keeps them internal. Messages that are not meant for publications are dropped immediately.

Platform publisher

The platform publisher is a piece of software that executes OAuth 2.0 requests on behalf of the users. It receives OAuth2.0 tokens from the Vault and interacts with supported third-party platform APIs to publish or send content as requested by the user. Messages it receives from the Gateway server are sent to Vault for decryption and then published.

Role:

This does the final publishing for users sending messages intended for their OAuth 2.0 accounts.

Bridge publisher

The Bridge publisher is a piece of software in the cloud that responsible for receiving bridge messages from the Gateway server, generates email aliases using SimpleLogin from the sender's phone number e.g. 237123456789@relaysms.me, and sends emails to the recipient specified in the payload. Replies from the recipients are encrypted by the Vault and sent via SMS to the sender where they are decrypted by the client.

Role:

The Bridge publisher provides bidirectional email communication over SMS by creating phone number based aliases via SimpleLogin, encrypts incoming replies through the Vault which are decrypted by the clients with no prior configuration needed.

Authentication and key exchange

Users create an account to enable them to store tokens (such as OAuth 2.0) so they can be used when messaging offline. Authentication (account creation and login) can be done using a RelaySMS client (currently officially supported are the Android[4] and iOS[5] clients).

The primary software on the server for storing OAuth 2.0 tokens, encryption states, phone number/ email and passwords is called **Vault**. Communication between clients and Vault (path 1 in figure 1.0) is done through gRPC[6]. The Vault encrypts all data at rest using AES-EAX with a 256 bits key; this includes encryption states, OAuth 2.0 tokens and operational files. Emails and phone numbers are hashed using HMAC-SHA512 with a 256-bit key and Argon2id is used for hashing the passwords.

Users are authenticated using an email or phone number and a password. Two-factor authentication is enabled to guarantee the email and phone number belongs to the user.

When the client is set up, it generates the following Curve25519 key pairs and nonce and sends to the Vault:

Client ID Key (CI_sig_pk): A long-term client key used for client identification purposes, deriving a long lived token (LLT) and Device ID. This key should be used only for signing purposes therefore an ed25519 derivative should be used.

Client Ratchet Key (eC_pk): An ephemeral key used to derive the Root Key for the client's Ratchet.

Client Header Key (eCH_pk): An ephemeral key used to initiate a Shared Secret with the server for encrypting the Ratchet's header.

Client Next Header Key (eCNH_pk): An ephemeral key used to initiate the next Shared secret with the server for encrypting the Ratchet's header.

Client Nonce (client_nonce): A random 128 bits value to be used as salt

The Vault generates the following Curve25519 key pairs and nonce and sends to client (path 2 in figure 1.0):

Server Ratchet Key (eS_pk): An ephemeral key used to init the Vault when publishing messages.

Server Header Key (eSH_pk): An ephemeral key used to initiate a Shared Secret with the client for encrypting the Ratchet's header.

Server Next Header Key (eSNH_pk): An ephemeral key used to initiate the next Shared secret with the client for encrypting the Ratchet's header.

Server Nonce (server_nonce): A random 128 bits value to be used as salt

Official clients have the Vault's long-term **Server Identity public key (SI_pk)** hardcoded. This pinned static key (s) is used to authenticate the server's ratchet keys (eS_pk) during shared-secret derivation for ratcheting, ensuring MITM protection.⁴

These keys are transmitted with a **128 bits Nonce** which would be used as **salt** when deriving a Root Key for the chain.

Long Lived Tokens (LLT)

Once the user has been authenticated, they receive a long lived token to be used for subsequent server queries. This includes request for:

- Getting names of platforms with saved OAuth 2.0 tokens
- Adding OAuth 2.0 tokens to account
- Revoking OAuth 2.0 tokens from account
- Deleting account

Refresh LLTs are generated on user authentication (creation, login and account recovery). This means users cannot be signed into more than 1 device at a time.⁵

Server generates the LLT as follows:

Secret = "good 256 bit shared secret"

```
payload = {  
    "eid": eid,  
    "iss": "https://smswithoutborders.com",  
    "iat": datetime.now(timezone.utc),  
    "exp": datetime.now(timezone.utc) + timedelta(days=3650),  
}
```

LLT = JWT().encode(

payload,

Secret,

alg="HS256"

⁴ Users self hosting Vault would need to generate and rebuild their clients replacing the official Vault keys with their generated copy.

⁵ Issuing a new LLT invalidates any previously issued LLTs

)

The client proves device ownership via proof-of-possession: it signs a canonical representation of each gRPC request, including method, nonce, and timestamp, using its device private key (**CI_sig**). The JWT is sent as Authorization: Bearer <LLT>, and the signature is sent as X-Sig.

The client therefore sends the following meta-data over their gRPC call:

```
request_string = (MethodName || Timestamp || Nonce) // Method has leading /  
signature = Sign(request_string, CI_sig)
```

Authorization: Bearer <LLT>

X-Sig-bin: <Signature>

X-Nonce-bin: <nonce>

X-Timestamp: <timestamp> // Unix seconds

The server verifies the request and drops it if verifications fail; the nonce is verified to guarantee the request is not being replayed. This is achieved by using a caching mechanism to hold the nonce for N-minutes (5 - 10 minutes) before deleting.

The server first validates the signature of the LLT using the server's JWT secret to confirm the token's integrity and authenticity. The user's ID (from the payload) is used to retrieve the user's Ed25519 public key (CI_sig_pk) from the database. The server then reconstructs the canonical request string by concatenating the gRPC method name (obtained from the request context), the timestamp, and the nonce (MethodName || Timestamp || Nonce), and uses the client's retrieved public key to verify the request signature. The nonce is checked and cached if it has not been used within the time window (or drops the request if used already). The checks involve finding the nonce in cache and verifying the incoming timestamp is greater than N-minutes of cached timestamp.

The server clears out nonces older than N-minutes.

Device ID (DID)

Device IDs serve as an alternative to phone-number-based verification. They are generated once a user authenticates with the Vault. The computed DID is then stored against the user's phone number in the encrypted Vault database.

Common use cases include:

Regional access limitations: When users cannot receive an SMS OTP, they can use a temporary number to receive the code and continue communications using their device ID. ⁶

Privacy-conscious publishing: Users can register with a trusted phone number but publish content using a temporary number. The device ID (a public, pseudonymous identifier) is concatenated to the SMS payload. ⁷

Device IDs are derived from the client identity public key (**CI_sig_pk**) as follows:

Message = "RelaySMS DID v1"

HO = HASH(Message || CI_sig_pk)

DID = Truncate(HO, 128–160 bits)

Truncating the SHA-256 hash to 128–160 bits ensures a negligible collision probability for practical user scales while maintaining a stable, deterministic, and public identifier. DIDs are public identifiers and are not intended to provide secrecy or authentication on their own.

The server references an associated account from the database with any incoming payload containing a DID. The acquired reference associated with a user's encryption state is used in an attempt to decrypt the incoming payload. This process fails if the retrieved encryption states were not initially authenticated by the entity referenced by the DID.

Encryption and data flow

Encryption

Encrypted messages are sent from RelaySMS clients and terminated on the Vault for online publishing. RelaySMS uses the Signal DR protocol for message encryption. The initial Root Key is derived via an

⁶ Email may be a more reliable global alternative.

⁷ See the threat model for RelaySMS for security considerations.

authenticated Diffie–Hellman key agreement between the client and the Vault. The handshake is based on *Noise IK* for authentication and forward secrecy while the Root Key derivation utilizes noise chaining.

Specifically, the Root Key is derived as follows:

Constants:

Salt = “RelaySMS v1”

Handshake_Salt = client_nonce || server_nonce

Info = "RelaySMS C2S DR v1"

Client DH computations:

DH1 = DH(eC, SI_pk) // authentication

DH2 = DH(eC, eS_pk) // forward secrecy

Server DH computations:

DH1 = DH(SI, eC_pk) // authentication

DH2 = DH(eS, eC_pk) // forward secrecy

Key derivation:

```
CK = HKDF(  
    salt = Salt,  
    input = Handshake_Salt,  
    info = Info  
)
```

```
CK = HKDF(  
    salt = CK,  
    input = DH1,  
    info = Info  
)
```

```
RK0 = HKDF(  
    salt = CK,  
    input = DH2,  
    info = Info  
)
```

```

        salt = CK,
        input = DH2,
        info = Info
    )

```

The Ratchet's Headers are also encrypted and their keys are derived similarly to the Root Key above. Specifically, the Header Keys are derived as follows:

Constants:

Salt = "RelaySMS v1"

Handshake_Salt = client_nonce || server_nonce

Info = "RelaySMS C2S DRHE v1"

Client Header Encryption DH computations:

DH1 = DH(eCH, SI_pk) // authentication

DH2 = DH(eCH, eSH_pk) // forward secrecy

Server Header Encryption DH computations:

DH1 = DH(SI, eCH_pk) // authentication

DH2 = DH(eSH, eCH_pk) // forward secrecy

Key derivation:

```

CK = HKDF(
    salt = Salt,
    input = Handshake_Salt,
    info = Info
)

```

```

CK = HKDF(
    salt = CK,
    input = DH1,

```

```

        info = Info
    )
    HRK0 = HKDF(
        salt = CK,
        input = DH2,
        info = Info
    )

```

Client Next Header Encryption DH computations:

```

DH1 = DH(eCNH, SI_pk) // authentication

DH2 = DH(eCNH, eSNH_pk) // forward secrecy

```

Server Next Header Encryption DH computations:

```

DH1 = DH(SI, eCNH_pk) // authentication

DH2 = DH(eSNH, eCNH_pk) // forward secrecy

```

Key derivation:

```

CK = HKDF(
    salt = Salt,
    input = Handshake_Salt,
    info = Info
)

CK = HKDF(
    salt = CK,
    input = DH1,
    info = Info
)

NHRK = HKDF(
    salt = CK,
    input = DH2,
    info = Info
)

```

)

Signal uses AES-256-CBC with PKCS#7 padding for message encryption. The header value containing the keys for the Ratchet chain is concatenated with the encrypted message. After each encryption, Signal advances the chain key to achieve forward secrecy. Other meta data such as payload versioning, bridge or platform publishing and device ID (if used) are concatenated with the encrypted message forming a final payload. The final payload is encoded in Base64 and transmitted over SMS to be forwarded to the Vault by a Gateway Client[9] (path 4, figure 1.0).

The AD used in encrypting the data is a Hash of the server's hardcoded static public key (SI_pk), the client's identity key (CI_sig_pk) and a fixed input. The derivation would as follows:

Client AD (AD) is derived as follows:

AD = HASH("RelaySMS AD v1" || CI_sig_pk || SI_pk)

Server's AD is derived as follows:

AD = HASH("RelaySMS AD v1" || SI_pk || CI_sig_pk)

Data flow

Storing OAuth 2.0 tokens (path 3, figure 1.0)

RelaySMS relies on OAuth2.0 tokens to publish on the user's behalf. Users choose a supported platform[10] to add their OAuth 2.0 token which is saved against their account on Vault. The OAuth2.0 scopes required for publishing is the 'write' scope. This aligns with the security principle of **least privilege**, ensuring applications only have the minimum access necessary for their function.

Publishing messages

Messages forwarded from the Gateway Client to the Gateway server (path 5, figure 1.0) include a header used in determining if a message is to be published with the user's OAuth 2.0 tokens (Platforms publishing - path 7, figure 1.0) or to be published with a phone number alias (Bridge publishing - path 8, figure 1.0). Either Platform or Bridge publication would send the encrypted payload to the Vault for

decryption (path 6, figure 1.0). The decrypted payload is parsed and sent to the service requested by the user.

If the user is using platforms and is authenticated by phone number - an SMS message with the result of a publication is sent back to the phone number used by the user for publishing (not the phone number they used for registration).

Failure is usually caused by:

- Error in content formation by client
- Issues with SDK (usually exceptions at implementation) - traceable by logs
- Revoked/expired OAuth 2.0 token
- Rejection by the target platform during publishing (e.g., invalid addresses, platform rate limits, or policy restrictions).

The specific nature of the failure is not included in the SMS message sent to the user.

Bridges

Bridges provides users the ability to send email messages online using an alias composed of their mobile phone number. The user's phone number is attached to a domain name and used as an address to send the email. Bridge messages do not expect the user to have any form of internet connection and are designed to be used as soon as a supported client is installed. Users receive an encrypted SMS when they receive a reply to their alias. This continues the DR session the client began when the first message was sent.

Bridges are configured to publish only from SMTP requests made by Gateway clients; this serves as a means of authenticating each incoming publish request. They are further configured to process only incoming messages from configured domain names. The incoming email is verified with its DKIM key and discarded if verification fails.

Official clients have the Vault's long-term **Server Identity public key (sS)** hardcoded. This pinned static key (s) is used to derive a shared-secret for ratcheting. It however differs from the more secure key generation methods when authenticating over the internet. This is due to the lack of a server ephemeral key and complete reliance on the pinned server identity public key.

The client generates a static identification key (cS) and sends the following Curve25519 keys and encrypted payloads to the server:

- **Client Ratchet Key (eC_pk):** An ephemeral public key used to root key derivation

- **Encrypted Identification Key (cS_pk_enc):** A long term key used to identify the client
- **Payload:** This is the initial message sent by the user to be published online. The initialized DR is used in encrypting this payload.

The Root Key is derived using a Noise IK pattern with handshake hashing. The very first message contains an encrypted payload - as the DR can be initiated using the derived Root Key. The subsequent messages maintain forward secrecy with the DR implementation.

Specifically, the Root Key is derived as follows:

Initializations:

Info = "RelaySMS C2S DR v1"

Header_info = "RelaySMS C2S DRHE v1"

$h = \text{HASH}(\text{"Noise_IK_25519_AESGCM_SHA256"})$

$CK = h$

$h = \text{HASH}(h \parallel sS_pk)$

$h = \text{HASH}(h \parallel eC_pk)$

Client Root Key (RK) derivation ($\rightarrow e, es, s, ss$) (Transport = SMS):

$DH_es = \text{DH}(eC, sS_pk)$

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_es, \text{info} = \text{Info})$

$cS_pk_enc = \text{ENCRYPT}(\text{key} = K, \text{input} = cS_pk, \text{ad} = h)^8 \text{ // send}$

$h = \text{HASH}(h \parallel cS_pk_enc)$

$DH_ss = \text{DH}(cS, sS_pk)$

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_ss, \text{info} = \text{Info})$

$\text{ciphertext} = \text{ENCRYPT}(\text{key} = K, \text{input} = "", \text{ad} = h)$

⁸ A zeroed 128 bits is can be used as nonce for the ENCRYPT methods when using AES-GCM

$h = \text{HASH}(h \parallel \text{ciphertext})$

$RK_0, HK, NHK = \text{HKDF}(\text{salt} = CK, \text{input} = DH_{ss}, \text{info} = \text{Header_info})$

Server Root Key (RK) derivation (Transport = SMS)

$DH_{es} = \text{DH}(sS, eC_{pk})$

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_{es}, \text{info} = \text{Info})$

$cS_{pk} = \text{DECRYPT}(\text{key} = K, \text{input} = cS_{pk_enc}, \text{ad} = h)$

$h = \text{HASH}(h \parallel cS_{pk_enc})$

$DH_{ss} = \text{DH}(sS, cS_{pk})$

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_{ss}, \text{info} = \text{Info})$

$\text{ciphertext} = \text{ENCRYPT}(\text{key} = K, \text{input} = "", \text{ad} = h)$

$h = \text{HASH}(h \parallel \text{ciphertext})$

$RK_0, HK, NHK = \text{HKDF}(\text{salt} = CK, \text{input} = DH_{ss}, \text{info} = \text{Header_info})$

Until the clients receive the server's ephemeral keys, they keep generating and sending new keys to the server. This is due to the unreliability of SMS messaging; it is impossible to know the very first message successfully made it to the server without receiving a server message⁹.

The server maintains the older states when a new one is received from the clients and increments the amount of authentications being done for the same phone number and appends that when creating an alias. For example, for $N > 1$ an alias would be: +12345_N_not_verified@relaysms.me. This serves to let the recipient know the authentication has changed for this alias (the sender).

The server will reply via SMS with its newly generated ephemeral keys (eS) for the very first message it receives. This serves as an authentication for the user and this would be their primary state for the alias going forward. This reply completes with (← e, ee, se) part of the IK handshake.

⁹ Doing this over SMS can be very costly and grows exponentially

Specifically the new root key is derived as follows:

Server Root Key (RK1) derivation

// continuing with the h derived during initial handshake

$h = \text{HASH}(h \parallel eS_pk)$

$DH_ee = DH(eS, eC_pk)$

// continuing with the CK derived during initial handshake

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_ee, \text{info} = \text{Info})$

$\text{ciphertext} = \text{ENCRYPT}(\text{key} = K, \text{input} = "", \text{ad} = h)$

$h = \text{HASH}(h \parallel \text{ciphertext})$

$DH_se = DH(sS, eC_pk)$

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_se, \text{info} = \text{Info})$

$\text{ciphertext1} = \text{ENCRYPT}(\text{key} = K, \text{input} = "", \text{ad} = h)$

$h = \text{HASH}(h \parallel \text{ciphertext})$

$RK_1, HK_1, NHK_1 = \text{HKDF}(\text{salt} = CK, \text{input} = "", \text{info} = \text{Header_info})$

Client Root Key (RK1) derivation

// continuing with the h derived during initial handshake

$h = \text{HASH}(h \parallel eS_pk)$

$DH_ee = DH(eC, eS_pk)$

// continuing with the CK derived during initial handshake

$CK, K = \text{HKDF}(\text{salt} = CK, \text{input} = DH_ee, \text{info} = \text{Info})$

ciphertext = ENCRYPT(key = K, input = "", ad = h)¹⁰

h = HASH(h || ciphertext)

DH_se = DH(eC, sS_pk)

CK, K = HKDF(salt = CK, input = DH_se, info = Info)

ciphertext1 = ENCRYPT(key = K, input = "", ad = h)

h = HASH(h || ciphertext)

RK₁, HK₁, NHK₁ = HKDF(salt = CK, input = "", info = Header_info)

The client deletes their states for RK0 once they have derived RK1. Then use RK1 in initializing a new Ratchet. Once the server receives the RK1 sessions it deletes the RK0 states and proceeds to initialize its Ratchet for RK1. This provides stronger security (with authentication and forward secrecy) for both parties going forward.

The clients should prompt the user to export and securely store their keypair; the exported file should be encrypted and a one-time recovery key provided to the user and never stored. This keypair would be primarily associated with their alias going forward and can be reimported into a new device when needed. When the alias receives a reply it is sent over SMS to the user. The message is encrypted with the primary state of the user; only the primary alias can receive a reply over SMS. The hash of the handshake is used as AD for the initial messages. The subsequent AD used when sending the messages are derived as follows:

Client AD is derived as follows:

AD = HASH("RelaySMS AD v1" || CI_pk || SI_pk)

Server's AD is derived as follows:

AD = HASH("RelaySMS AD v1" || SI_pk || CI_pk)

¹⁰ In case the ciphertext from the server is received, it should be decrypted in the next step and throws an exception in case it fails. This would enable the developer know what failed and where before moving on with the key derivation and decryption

Limitations

Users send messages one way for the most part; replies are only received by SMS when they are related to Bridges. This means the Ratchet keeps spinning one way until the user gets a bridge reply. One way of solving this includes sending back Ratchet key messages in the confirmations sent to the user. This is just a temporary solution (that increases the cost of SMS messaging for the project) because there is very little guarantee the user would put the keys into their app.¹¹

A workaround for this would be to spin the ratchet for the user once they come online; this can be done by encrypting and sending a confirmation history to the user (messages that successfully or failed to publish).

Conclusions

The constraints we face from the threat models SMS presents to the acceptable size of an SMS payload makes this an interesting problem to solve. We continue to bring solutions to the problem, while aiming to make this more secure, private and cheaper for the users to use. The majority of protocols and standardization we use in this document were built up from very naive¹² first implementations. The implementation as we have here is currently ongoing a security audit.

¹¹ *If this is sent back by SMS, the users would have to paste the SMS message into their RelaySMS app. There is very little guarantee the user would be able to do that.*

¹² *As software engineers, we can implement what can be secure but not knowledged on the right practices for implementing them. This is an endorsement for security focused softwares to have security engineers audit and work on the project - should be as critical as requiring developers for adding features.*

References:

[TODO: Finish inserting references]